





06-1 문자열 함수

- 처음 알아볼 SQL 함수는 문자열 함수로, 문자열 함수는 문자열 연결, 형 변환, 공백 제거, 치환 등에서 다양하게 사용함
- 문자열 함수는 문자열뿐만 아니라 열 이름을 인자로 활용할 수 있음

■ 문자열과 문자열을 연결하는 함수 — CONCAT

- 문자열과 문자열은 CONCAT 함수를 사용해 연결함
- CONCAT 함수의 사용 방법은 연결할 문자열을 괄호 안에 쉼표로 구분하여 함수의 인자로 나열하면 됨
- 단어가 아닌 열 이름을 나열할 때도 마찬가지임

CONCAT 함수 예

```
SELECT CONCAT('I ', 'Love ', 'MySQL') AS col_1;
```



06-1 문자열 함수

■ 문자열과 문자열을 연결하는 함수 — CONCAT

1. 다음은 열 이름과 문자열을 인자로 전달하는 쿼리임.

Do it! CONCAT 함수로 열 이름과 문자열 연결

```
SELECT CONCAT(first_name, ', ', last_name) AS customer_name FROM  
customer;
```

결과를 살펴보면 first_name과 last_name 열을 연결할 때
이를 구분하기 위해 중간에 문자열인 ', '를 사용함.

이렇게 열을 합치면서 필요한 문자열도 함께 사용할 수 있음

실행 결과

customer_name
MARY, SMITH
PATRICIA, JOHNSON
LINDA, WILLIAMS
BARBARA, JONES
ELIZABETH, BROWN
JENNIFER, DAVIS
MARIA, MILLER
SUSAN, WILSON
MARGARET, MOORE
DOROTHY, TAYLOR
LISA, ANDERSON
NANCY, THOMAS
KAREN, JACKSON
BETTY, WHITE
HELEN, HARRIS



06-1 문자열 함수

■ 문자열과 문자열을 연결하는 함수 — CONCAT

2. 그런데 여러 열을 합칠 때
모든 열 이름 사이에 직접 일일이 쉼표를 입력하기는 번거로움.

이때 CONCAT_WS를 사용하면
구분자를 미리 정의해서 자동으로 적용할 수 있음.

다음과 같이 첫 번째 인수에
나머지 인수를 구분하기 위한 기호를 넣어 보자



06-1 문자열 함수

■ 문자열과 문자열을 연결하는 함수 — CONCAT

2.

Do it! CONCAT_WS 함수로 구분자 지정

```
SELECT CONCAT_WS(' ', first_name,  
last_name, email) AS customer_name FROM  
customer;
```

실행 결과를 살펴보면
연결할 문자열 사이에
구분 기호()가 추가된 것을
확인할 수 있음

실행 결과

customer_name
MARY, SMITH, MARY.SMITH@sakilacustomer.org
PATRICIA, JOHNSON, PATRICIA.JOHNSON@sakilacustomer.org
LINDA, WILLIAMS, LINDA.WILLIAMS@sakilacustomer.org
BARBARA, JONES, BARBARA.JONES@sakilacustomer.org
ELIZABETH, BROWN, ELIZABETH.BROWN@sakilacustomer.org
JENNIFER, DAVIS, JENNIFER.DAVIS@sakilacustomer.org
MARIA, MILLER, MARIA.MILLER@sakilacustomer.org
SUSAN, WILSON, SUSAN.WILSON@sakilacustomer.org
MARGARET, MOORE, MARGARET.MOORE@sakilacustomer.org
DOROTHY, TAYLOR, DOROTHY.TAYLOR@sakilacustomer.org
LISA, ANDERSON, LISA.ANDERSON@sakilacustomer.org
NANCY, THOMAS, NANCY.THOMAS@sakilacustomer.org
KAREN, JACKSON, KAREN.JACKSON@sakilacustomer.org
BETTY, WHITE, BETTY.WHITE@sakilacustomer.org
HELEN, HARRIS, HELEN.HARRIS@sakilacustomer.org



06-1 문자열 함수

■ 문자열과 문자열을 연결하는 함수 — CONCAT

3. 만약 인수로 NULL을 입력할 경우 결과는 모두 NULL이 반환됨.
이는 NULL이 어떠한 문자열과 결합하거나 계산되더라도 결과가 NULL이기 때문임

Do it! CONCAT 함수로 NULL과 열 이름 연결

```
SELECT CONCAT(NULL, first_name, last_name) AS customer_name FROM  
customer;
```

실행 결과	
	customer_name
▶	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL
	NULL



06-1 문자열 함수

■ 문자열과 문자열을 연결하는 함수 — CONCAT

4. CONCAT_WS는 비어 있는 열을 그대로 적용함.
그러나 구분 인수 뒤에 오는 NULL은 무시하여 건너뛴.

그래서 합치려는 열에 NULL이 있을 경우
결과가 NULL이 아닌 NULL을 제외한 결합 문자가 출력됨

Do it! CONCAT_WS 인자로 NULL이 있는 경우

```
SELECT CONCAT_WS(' ', first_name, NULL,  
last_name) as customer_name FROM customer;
```

실행 결과

customer_name
MARY, SMITH
PATRICIA, JOHNSON
LINDA, WILLIAMS
BARBARA, JONES
ELIZABETH, BROWN
JENNIFER, DAVIS
MARIA, MILLER
SUSAN, WILSON
MARGARET, MOORE
DOROTHY, TAYLOR
LISA, ANDERSON
NANCY, THOMAS
KAREN, JACKSON
BETTY, WHITE
HELEN, HARRIS



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

- 숫자를 날짜로 또는 날짜를 숫자로 변환하는 등 데이터를 다양한 형태로 변환해야 하는 경우가 있음

- 사용자가 임의로 형 변환하는 과정을 명시적 형 변환이라고 하는데,

이때 형 변환을 위해 사용하는 함수로는 CAST, CONVERT가 있음



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

- 다음은 CAST와 CONVERT 함수의 기본 형식임
- 함수 모두 데이터 유형을 변환하는데 사용되며 사용법도 비슷하지만, CONVERT 함수는 CAST 함수와 달리 문자열 집합을 다른 문자열 집합으로 변환할 수 있음

CAST 함수의 기본 형식

```
SELECT CAST(열 AS 데이터 유형) FROM 테이블;
```

CONVERT 함수의 기본 형식

```
SELECT CONVERT(열, 데이터 유형) FROM 테이블;
```



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

- 그런데 CAST와 CONVERT를 이용해
형 변환을 할 수 있는 데이터 유형은 한정적임

CAST 및 CONVERT에 사용 가능한 데이터 유형

BINARY	CHAR
DATE	DATETIME
TIME	DECIMAL
JSON	NCHAR
SIGNED[INTEGER]	UNSIGNED[INTEGER]



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

1. CAST 함수를 활용해 문자열 '2'라는 글자를 부호가 없는 정수형 2로 변경함

Do it! 문자열을 부호 없는 정수형으로 변경

```
SELECT  
  4 / '2',  
  4 / 2,  
  4 / CAST('2' AS UNSIGNED);
```

실행 결과

	4 / '2'	4 / 2	4 / CAST('2' AS UNSIGNED)
▶	2	2.0000	2.0000



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

1. 숫자 4와 문자열 2를 나누어 보면
정수로 2라는 결과가 반환됨.

그 다음, 숫자 4와 숫자 2를 나누었더니
2.0000으로 실수로 반환됨.

마지막엔 숫자 4와 문자 2를 나눌 때
문자 '2'의 값을 USINGED로 명시적 형 변환을 했더니
결곶값이 실수로 표현된 것을 확인할 수 있음



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

2. CAST 함수를 이용해
날짜형 데이터를 숫자형으로 변환해 보자.

먼저, 다음과 같이 NOW 함수를 활용해
현재 날짜와 시간을 가져와 출력해 보자

Do it! NOW 함수로 현재 날짜와 시간 출력

```
SELECT NOW();
```

실행 결과

NOW()
2023-08-20 13:29:46



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

2. 이번에는 CAST 함수를 이용해
NOW 함수로 가져온 값을 정수형으로 변환해 보자.

Do it! CAST 함수로 날짜형을 숫자형으로 변환

```
SELECT CAST(NOW() AS SIGNED);
```

실행 결과

CAST(NOW() AS SIGNED)
▶ 20230820133013

NOW 함수의 실행 결과와 비교해 보면
날짜와 시간을 구분하는 기호 없이
날짜가 숫자 형식으로 이루어진 데이터가 출력됨



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

3. 이번에는 반대로 숫자형을 날짜형 또는 문자열로 변환해 보자.
마찬가지로 CAST 함수를 활용함

Do it! CAST 함수로 숫자형을 날짜형으로 변환

```
SELECT CAST(20230819 AS DATE);
```

실행 결과

CAST(20230819 AS DATE)
▶ 2023-08-19

Do it! CAST 함수로 숫자형을 문자열로 변환

```
SELECT CAST(20230819 AS CHAR);
```

실행 결과

CAST(20230819 AS CHAR)
▶ 20230819



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

4. 이제 CONVERT 함수를 알아보자.
CONVERT 함수는 다음과 같은 형태로 인자 2개를 넘겨 사용함.

CAST 함수와 사용법이 거의 비슷하지만
CONVERT에서는 CAST에서는 지원하지 않는 스타일을 정의할 수 있음.

다음 쿼리는 앞서 CAST 함수를 사용한 실습에서
CAST 대신 CONVERT를 사용한 것임



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

4. CAST 함수를 사용했을 때와 마찬가지로 날짜와 시간을 구분하는 기호 없이 숫자로만 이루어진 데이터가 출력됨

Do it! CONVERT 함수로 날짜형을 정수형으로 변환

```
SELECT CONVERT(NOW(), SIGNED);
```

실행 결과

CONVERT(now(), SIGNED)
20230820133950



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

6. CAST, CONVERT 함수를 사용할 때
AS CHAR(5) 또는 CHAR(5)와 같이 문자열의 길이를 지정할 수도 있음.

문자열 길이를 지정하면 문자열을 변환할 때
CHAR(5)로 지정한 값보다 문자열 길이가 작으면 문자열이 잘려 출력됨.

다음은 숫자 20230819를 문자열로 변환하되 문자열 길이를 5로 지정한 쿼리임.
결과를 보면 문자열이 5개까지만 출력된 것을 확인할 수 있음

Do it! CHAR로 데이터 길이 지정

```
SELECT CONVERT(20230819, CHAR(5));
```

실행 결과

	CONVERT(20230819, CHAR(5))
▶	20230



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

7. 이번에는 엄청난 큰 수에 1을 더하는 쿼리를 작성해 보자.
이를 실행하면 가장 큰 숫자형(BIGINT)의 범위를 넘어 오버플로가 발생함.

Do it! 오버플로 발생 예

```
SELECT 9223372036854775807 + 1;
```

오버플로로 인한 오류를 살펴보면 입력한 식이
BIGINT 값의 범위를 초과했다는 내용이 적혀 있음.

```
Error Code: 1690. BIGINT value is out of range in '(9223372036854775807  
+ 1)'
```



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

7. 이런 오버플로를 예방하고 싶다면
CAST 함수를 활용해 입력값을 UNSIGNED로 변경하여 연산하면 됨.

Do it! CAST 함수로 오버플로 방지

```
SELECT CAST(9223372036854775807 AS UNSIGNED) + 1;
```

실행 결과

	CAST(9223372036854775807 AS UNSIGNED) + 1
▶	9223372036854775808

이와 같이 9223372036854775807에 1을 더해도
오버플로 없이 출력되는 것을 확인할 수 있음



06-1 문자열 함수

■ 데이터 형 변환 함수 — CAST, CONVERT

8. CAST 대신 CONVERT 함수를 넣어 출력해 보자.
실행 결과는 같음

Do it! CONVERT 함수로 오버플로 방지

```
SELECT CONVERT(9223372036854775807, UNSIGNED) + 1;
```

실행 결과

	CONVERT(9223372036854775807,UNSIGNED) + 1
▶	9223372036854775808



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

- NULL은 어떠한 연산 작업을 진행해도 NULL이 반환되므로 NULL 때문에 예기치 못한 결과를 얻게 되는 경우가 종종 있음
- 그래서 테이블에 NULL이 있는 경우 문자열 또는 숫자로 데이터를 바꾸는 것이 좋음
- 이와 같이 NULL을 치환하기 위해 IFNULL 함수를 사용함
- 이 함수를 사용하면 NULL을 대체할 다른 값으로 변환함



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

- IFNULL 함수는 인자로 열 이름과 대체할 값을 전달하면 해당 열에 있는 NULL을 대체할 값으로 변환함

IFNULL 함수의 기본 형식

IFNULL(열, 대체할 값)



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

- COALESCE 함수도 NULL을 대체하는데
COALESCE 함수의 경우 여러 열 이름을 인자로 전달함.

COALESCE 함수의 기본 형식

COALESCE(열 1, 열 2, ...)

이와 같이 IFNULL의 경우에는 NULL을 변환하기 위해
대체할 값 또는 열 한 개만 인자로 사용이 가능하지만,

COALESCE는 NULL이 아닌 값이 나올 때까지
후보군의 여러 열을 입력할 수 있음



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

1. 우선 실습을 위해 다음과 같이 doit_null이라 테이블을 생성하는 쿼리를 작성해 보자. 이 테이블에는 NULL이 존재함

Do it! 테이블 생성

```
CREATE TABLE doit_null (  
  col_1 INT,  
  col_2 VARCHAR(10),  
  col_3 VARCHAR(10),  
  col_4 VARCHAR(10),  
  col_5 VARCHAR(10)  
);  
  
INSERT INTO doit_null VALUES (1, NULL, 'col_3', 'col_4', 'col_5');  
INSERT INTO doit_null VALUES (2, NULL, NULL, NULL, 'col_5');  
INSERT INTO doit_null VALUES (3, NULL, NULL, NULL, NULL);  
  
SELECT * FROM doit_null;
```

실행 결과

	col_1	col_2	col_3	col_4	col_5
▶	1	NULL	col_3	col_4	col_5
	2	NULL	NULL	NULL	col_5
	3	NULL	NULL	NULL	NULL



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

1. 먼저 col_2 열의 값이 NULL이면 IFNULL 함수를 사용해 공백("")으로 대체해 보자.

Do it! IFNULL 함수로 col_2열의 NULL 대체

```
SELECT col_1, IFNULL(col_2, "") AS col_2, col_3, col_4, col_5  
FROM doit_null WHERE col_1 = 1;
```

실행 결과

	col_1	col_2	col_3	col_4	col_5
▶	1		col_3	col_4	col_5

테이블을 생성했을 때와 비교해 보면
1행에서 col_2 열에 해당하는 값이 공백으로 출력되었음을 확인할 수 있음.

이 쿼리는 col_1 ~ col_5까지 검색하는데,
col_2 열 데이터가 NULL이면 공백으로 표시한 결과를 보여 줌



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

3. 이번에는 col_2열의 값이 NULL이면 col_3 열의 값으로 대체해 보자.

Do it! IFNULL 함수로 col_3열의 NULL 대체

```
SELECT col_1, IFNULL(col_2, col_3) AS col_2, col_3, col_4, col_5  
FROM doit_null WHERE col_1 = 1;
```

실행 결과

	col_1	col_2	col_3	col_4	col_5
▶	1	col_3	col_3	col_4	col_5

이번에는 col_2 열의 데이터가 col_3 열에 있던
col_3 데이터로 대체된 것을 확인할 수 있음.

앞선 실습에서는 사용자가 지정한 문자를 사용해 NULL을 대체했지만
여기서는 다른 열로 NULL을 대체해 본 것임



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

4. 이제 COALESCE 함수를 사용해 보자.

COALESCE 함수는 첫 번째 인자로 전달한 열에 NULL이 있을 때 그 다음 인자로 작성한 열의 데이터로 대체함.

만약 이 함수에 N개의 인자를 작성했다면 순차로 대입함.



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

4. 다음 쿼리는 col_2의 값이 NULL일 때
다음 인자인 col_3의 데이터도 NULL이면
그다음 인자인 col_4의 데이터를 확인하여 대입함

Do it! COALESCE 함수로 NULL을 다른 데이터로 대체: 마지막 인자에 데이터가 있는 경우

```
SELECT col_1, COALESCE(col_2, col_3, col_4, col_5)  
FROM doit_null WHERE col_1 = 2;
```

실행 결과

col_1	COALESCE(col_2, col_3, col_4, col_5)
3	col_5



06-1 문자열 함수

■ NULL을 대체하는 함수 — IFNULL, COALESCE

5. 하지만 만약 마지막 인자까지도
NULL이 저장되어 있다면 결국 NULL을 반환함

Do it! COALESCE 함수로 NULL을 다른 데이터로 대체: 마지막 인자에도 NULL이 있는 경우

```
SELECT col_1, COALESCE(col_2, col_3, col_4, col_5)
FROM doit_null WHERE col_1 = 3;
```

실행 결과

col_1	COALESCE(col_2, col_3, col_4, col_5)
3	NULL



06-1 문자열 함수

■ 소문자 혹은 대문자로 변경하는 함수 — LOWER, UPPER

1. LOWER 함수는 대문자를 소문자로, UPPER 함수는 소문자를 대문자로 변경함.
사용 방법은 매우 간단하므로 바로 실습을 통해 확인해 보자.

Do it! LOWER 함수로 소문자를, UPPER 함수로 대문자로 변경

```
SELECT 'Do it! SQL', LOWER('Do it! SQL'), UPPER('Do it! SQL');
```

실행 결과

	Do it! SQL	LOWER('Do it! SQL')	UPPER('Do it! SQL')
▶	Do it! SQL	do it! sql	DO IT! SQL

결과에서 알 수 있듯 LOWER 함수는 'Do it! SQL'을 모두 소문자로,
UPPER 함수는 모두 대문자로 변경함



06-1 문자열 함수

■ 소문자 혹은 대문자로 변경하는 함수 — LOWER, UPPER

- 이번에는 문자열 대신 열 이름을 입력해 보자.
해당 열의 데이터들이 소문자 또는 대문자로 변경되어 출력된 것을 확인할 수 있음

Do it! LOWER 함수로 소문자를, UPPER 함수로 대문자로 변경

```
SELECT email, LOWER(email), UPPER(email) FROM customer;
```

실행 결과

email	LOWER(email)	UPPER(email)
▶ MARY.SMITH@sakilacustomer.org	mary.smith@sakilacustomer.org	MARY.SMITH@SAKILACUSTOMER.ORG
PATRICIA.JOHNSON@sakilacustomer.org	patricia.johnson@sakilacustomer.org	PATRICIA.JOHNSON@SAKILACUSTOMER.ORG
LINDA.WILLIAMS@sakilacustomer.org	linda.williams@sakilacustomer.org	LINDA.WILLIAMS@SAKILACUSTOMER.ORG



06-1 문자열 함수

■ 공백을 제거하는 함수 — LTRIM, RTRIM, TRIM

- 사용자가 어떤 데이터를 입력할 때 공백을 입력하는 경우가 있는데, 데이터 관리자에게는 공백이 문제를 일으킬 수 있으므로 관리 대상임
- 예를 들어 사용자가 회원 가입을 할 때 실수로 아이디나 비밀번호 뒤에 의도하지 않은 공백을 입력했고 데이터베이스에서 공백을 허용했다면

이후 사용자가 로그인할 때 아이디, 비밀번호를 제대로 입력해도 공백이 입력이 되지 않아 인증 처리가 제대로 되지 않을 것임



06-1 문자열 함수

■ 공백을 제거하는 함수 — LTRIM, RTRIM, TRIM

- 이와 같이 실수로 공백을 입력하더라도 데이터가 공백 없이 저장되도록 공백을 제거하는 LTRIM, RTRIM, TRIM 함수를 사용해 보자
- LTRIM 함수의 L은 왼쪽을, RTRIM 함수의 R은 오른쪽을 의미하며 각각 문자열의 왼쪽(앞) 또는 오른쪽(뒤)의 공백을 제거함
- 만약 양쪽 공백을 모두 제거하려면 TRIM 함수를 사용하면 됨



06-1 문자열 함수

■ 공백을 제거하는 함수 — LTRIM, RTRIM, TRIM

1. 다음과 같이 LTRIM을 사용해 'Do it!' 문자열 앞 공백을 삭제해 보자.
결과 왼쪽 열과 비교해 보면 차이를 확인해 볼 수 있음

Do it! LTRIM 함수로 왼쪽 공백 제거

```
SELECT ' Do it! MySQL', LTRIM(' Do it! MySQL');
```

실행 결과

	Do it! MySQL	LTRIM(' Do it! MySQL')
▶	Do it! MySQL	Do it! MySQL



06-1 문자열 함수

■ 공백을 제거하는 함수 — LTRIM, RTRIM, TRIM

2. 이번에는 RTRIM을 사용해 'Do it!' 문자열 뒤 공백을 삭제해 보자. 마찬가지로 결과 왼쪽열과 비교해 보면 차이를 확인해 볼 수 있음

Do it! RTRIM 함수로 오른쪽 공백 제거

```
SELECT 'Do it! MySQL ', RTRIM('Do it! MySQL ');
```

실행 결과

Do it! MySQL	RTRIM('Do it! MySQL ')
▶ Do it! MySQL	Do it! MySQL



06-1 문자열 함수

■ 공백을 제거하는 함수 — LTRIM, RTRIM, TRIM

3. 이번에는 TRIM 함수를 사용해 문자열의 양쪽 공백을 모두 제거해 보자

Do it! TRIM 함수로 양쪽 공백 제거

```
SELECT ' Do it! MySQL ', TRIM(' Do it! MySQL ');
```

실행 결과

Do it! MySQL	TRIM(Do it! MySQL)
▶ Do it! MySQL	Do it! MySQL



06-1 문자열 함수

■ 공백을 제거하는 함수 — LTRIM, RTRIM, TRIM

4. 그런데 TRIM 함수는 공백이 아닌
단어 앞뒤에 있는 특정 문자를 제거하는 기능도 있음.
다음 쿼리를 통해 문자열 양 끝에 있는 '#'을 삭제해 보자

Do it! TRIM 함수로 양쪽 문자 제거

```
SELECT TRIM(BOTH '#' FROM '# Do it! MySQL #');
```

실행 결과

	TRIM(BOTH '#' FROM '# Do it! SQL #')
▶	Do it! SQL



06-1 문자열 함수

■ 문자열 크기 또는 개수를 반환하는 함수 — LENGTH, CHAR_LENGTH

1. 영어 문자열과 한글 문자열의 바이트 크기를 확인할 수 있음.

Do it! LENGTH 함수로 문자열의 크기 반환

```
SELECT LENGTH('Do it! MySQL'), LENGTH('두잇  
마이애스큐엘');
```

실행 결과

LENGTH('Do it! MySQL')	LENGTH('두잇 마이애스큐 엘')
12	25

LENGTH 함수를 사용해 'Do it! MySQL'의 문자열 크기로는 12바이트를,
'두잇 마이애스큐엘'의 문자열 크기로는 25바이트를 반환한 것을 확인할 수 있음



06-1 문자열 함수

■ 문자열 크기 또는 개수를 반환하는 함수 — LENGTH, CHAR_LENGTH

2. 다음 실습으로 영어 및 공백은 1바이트, 한글과 한자는 3바이트, 특수문자는 3바이트를 사용함을 확인할 수 있음

Do it! LENGTH 함수로 다양한 문자의 크기 반환

```
SELECT LENGTH('A'), LENGTH('강'), LENGTH('漢'), LENGTH('<'),  
LENGTH(' ');
```

실행 결과

	LENGTH('A')	LENGTH('강')	LENGTH('漢')	LENGTH('<')	LENGTH(' ')
▶	1	3	3	3	1



06-1 문자열 함수

■ 문자열 크기 또는 개수를 반환하는 함수 — LENGTH, CHAR_LENGTH

3. 바이트 크기로는 문자열의 개수를 정확히 알기 어려움.
문자열 개수를 확인하고 싶다면 CHAR_LENGTH 함수를 사용하면 됨.

Do it! CHAR_LENGTH 함수로 문자열의 개수 반환

```
SELECT CHAR_LENGTH('Do it! MySQL'), CHAR_LENGTH('두잇  
마이애스큐엘');
```

실행 결과

CHAR_LENGTH('Do it! MySQL')	CHAR_LENGTH('두잇 마이애스큐 엘')
12	9

CHAR_LENGTH 함수를 사용해 'Do it! MySQL'의 문자열 개수로는
띄어쓰기를 포함해 12, '두잇 마이애스큐엘'은 9를 반환한 것을 확인할 수 있음



06-1 문자열 함수

■ 문자열 크기 또는 개수를 반환하는 함수 — LENGTH, CHAR_LENGTH

4. LENGTH와 CHAR_LENGTH 함수에 문자열 대신 열 이름을 인수로 전달할 수도 있음.
여기서는 custom 테이블의 first_name 열 이름을 입력해 보자.

Do it! LENGTH와 CHAR_LENGTH 함수에 열 이름 전달

```
SELECT first_name, LENGTH(first_name), CHAR_LENGTH(first_name) FROM customer;
```

실행 결과

first_name	LENGTH(first_name)	CHAR_LENGTH(first_name)
CHRISTINE	9	9
MARIE	5	5
JANET	5	5
CATHERINE	9	9
FRANCES	7	7
ANN	3	3
JOYCE	5	5
DIANE	5	5
ALICE	5	5
JULIE	5	5

와 같이 열 이름을 넣어 실행하면
행에 저장된 문자열의 크기와 개수를 반환함



06-1 문자열 함수

■ 특정 문자까지의 문자열 길이를 반환하는 함수 — POSITION

1. 다음 쿼리는 느낌표가 있는 문자까지의 문자열 길이를 반환함.

Do it! POSITION 함수로 특정 문자까지의 크기 반환

```
SELECT 'Do it!! SQL', POSITION('!' IN 'Do it!! MySQL');
```

실행 결과

Do it!! SQL	POSITION('!' IN 'Do it!! SQL')
▶ Do it!! SQL	6

쿼리를 실행한 결과,
첫 번째 느낌표까지 해서 문자열 길이로
6을 반환한 것을 확인할 수 있음



06-1 문자열 함수

■ 특정 문자까지의 문자열 길이를 반환하는 함수 — POSITION

2. POSITION 함수는 지정한 문자가 탐색 대상이 되는 문자열에 존재하지 않으면 0을 반환함

Do it! 탐색 문자가 없는 경우

```
SELECT 'Do it!! SQL', POSITION('#' IN 'Do it!! MySQL');
```

실행 결과

	Do it!! SQL	POSITION('#' IN 'Do it!! SQL')
▶	Do it!! SQL	0



06-1 문자열 함수

■ 지정한 길이만큼 문자열을 반환하는 함수 — LEFT, RIGHT

- LEFT 함수는 문자열의 왼쪽부터,
RIGHT 함수는 오른쪽부터 정의한 위치만큼의 문자열을 반환함
- 다음과 같이 'Do it SQL'이라는 문장이 있다면 문자열을 셀 때 시작값은 1임

1	2	3	4	5	6	7
D	O	I	T	S	Q	L



06-1 문자열 함수

■ 지정한 길이만큼 문자열을 반환하는 함수 — LEFT, RIGHT

- 다음은 LEFT와 RIGHT 함수를 사용해 주어진 문자열 왼쪽에서 2개, 오른쪽에서 2개의 문자 열을 반환하는 쿼리임
- 이때 함수의 첫 번째 인자로는 문자열을, 두 번째 인자로는 숫자를 입력함

Do it! LEFT와 RIGHT 함수로 왼쪽과 오른쪽 2개의 문자열 반환

```
SELECT 'Do it! MySQL', LEFT('Do it! MySQL', 2), RIGHT('Do it! MySQL', 2);
```

실행 결과

	Do it! SQL	LEFT('Do it! SQL', 2)	RIGHT('Do it! SQL', 2)
▶	Do it! SQL	Do	QL

- 쿼리를 실행한 결과, LEFT 함수에 의해 문장 앞에서부터 1~2번째 문자열인 'Do'를 반환하고
RIGHT 함수에 의해 문장 뒤에서부터 1~2번째 문자열인 'QL'을 반환함



06-1 문자열 함수

■ 지정한 범위의 문자열을 반환하는 함수 — SUBSTRING

- SUBSTRING 함수는 지정한 범위의 문자열을 반환함
- 함수의 2번째 인자에는 시작 위치를, 3번째 인자에는 시작 위치로부터 반환할 문자열 개수를 입력함

1. 다음은 4번째 문자부터 문자 2개를 반환하는 쿼리임

Do it! SUBSTRING 함수로 지정한 범위의 문자열 반환

```
SELECT 'Do it! MySQL', SUBSTRING('Do it! MySQL', 4, 2);
```

실행 결과

Do it! SQL	SUBSTRING('Do it! SQL', 4, 2)
▶ Do it! SQL	it

함수를 실행하면 'Do it! MySQL'의 4번째 위치에 있는 문자열인 i부터 시작해 2개의 문자열을 반환해야 하므로 결과로 it이 출력된 것을 확인할 수 있음



06-1 문자열 함수

■ 지정한 범위의 문자열을 반환하는 함수 — SUBSTRING

2. SUBSTRING 함수도 다른 함수처럼 열 이름을 인수로 전달하여 사용할 수도 있음.
다음은 first_name 열에 저장된 데이터의 2번째 문자부터 문자 3개를 반환하는 쿼리임.
결과값을 비교하기 위해 원래 문자도 조회하도록 쿼리를 작성해 보자.

Do it! SUBSTRING 함수에 열 이름 전달

```
SELECT first_name, SUBSTRING(first_name, 2, 3) FROM customer;
```

실행 결과

first_name	SUBSTRING(first_name, 2, 3)
MARY	ARY
PATRICIA	ATR
LINDA	IND
BARBARA	ARB
ELIZABETH	LIZ
JENNIFER	ENN
MARIA	ARI
SUSAN	USA
MARGARET	ARG

이와 같이 열 이름을 넣어 실행하면 실행 결과의 두 번째 열을 통해 문자열의 2번째 문자부터 시작해 3개의 문자열을 반환한 것을 확인할 수 있음



06-1 문자열 함수

■ 지정한 범위의 문자열을 반환하는 함수 — SUBSTRING

3. SUBSTRING 함수는 POSITION 함수와 함께 사용할 수도 있음.
다음은 POSITION 함수로 @ 문자 위치를 계산한 다음,
SUBSTRING 함수에 사용하여 @ 문자 바로 앞까지의 문자열을 조회하는 쿼리임.

Do it! SUBSTRING과 POSITION 함수 조합

```
SELECT SUBSTRING('abc@email.com', 1, POSITION('@' IN  
'abc@email.com') - 1);
```

실행 결과

SUBSTRING('abc@email.com', 1, POSITION('@' IN 'abc@email.com') - 1)
▶ abc

문자열의 1번째 문자부터 시작해 POSITION 함수를 통해
@까지의 문자열 크기를 반환해 그 수에서 -1을 더한 크기만큼
즉 3개의 문자열을 반환한 결과로 abc를 출력함



06-1 문자열 함수

■ 특정 문자를 다른 문자로 대체하는 함수 — REPLACE

- REPLACE 함수는 이름만 봐도 알 수 있듯이 지정한 문자를 다른 문자로 대체함
- 다음은 문자 A를 C로 대체하는 쿼리임.
customer 테이블의 first_name 열에서 A로 시작하는 데이터를 조회해
이 데이터에 있는 문자열 A를 C로 대체해 보자

Do it! REPLACE 함수로 문자 변경

```
SELECT first_name, REPLACE(first_name, 'A', 'C')  
FROM customer WHERE first_name LIKE 'A%';
```

실행 결과

first_name	REPLACE(first_name, 'A', 'C')
ANGELA	CNGELC
AMY	CMY
ANNA	CNNC
AMANDA	CMCNDC
ANIN	CNN
ALICE	CLICE
ASHLEY	CSHLEY
ANDREA	CNDREC
ANNE	CNNE



06-1 문자열 함수

■ 문자를 반복하는 함수 — REPEAT

- REPEAT 함수는 지정한 문자를 반복할 때 사용함
- 이때, 반복할 문자와 반복 횟수를 인자로 전달하면 됨

1. 다음은 문자 0을 10번 반복하여 출력하는 쿼리임.

Do it! REPEAT 함수로 문자 반복

```
SELECT REPEAT('0', 10);
```

실행 결과

REPEAT('0', 10)
▶ 0000000000

쿼리를 실행한 결과, 이와 같이 문자열 0을 10번 반복해 출력한 것을 확인할 수 있음



06-1 문자열 함수

■ 문자를 반복하는 함수 — REPEAT

2. REPEAT 함수도 다른 함수와 조합하여 사용할 수 있음.
다음은 REPLACE 함수를 함께 사용하여 문자 A를 C로 바꾸되
대체하는 문자를 10번 반복해 출력하는 쿼리임.

Do it! REPEAT과 REPLACE 함수 조합

```
SELECT first_name, REPLACE(first_name, 'A', REPEAT('C', 10))  
FROM customer WHERE first_name LIKE '%A%';
```

%A%를 사용해 A가 포함된 모든 데이터를 조회한 뒤,
문자 A를 10개의 C로 대체한 것을 확인할 수 있음

실행 결과

first_name	REPLACE(first_name, 'A', REPEAT('C', 10))
MARY	MCCCCCCCCCRY
PATRICIA	PCCCCCCCCCTRICCCCCCCCC
LINDA	LINDCCCCCCCC
BARBARA	BCCCCCCCCCRBCCCCCCCCRCRCCCCCCCC
ELIZABETH	ELIZCCCCCCCCCBETH
MARIA	MCCCCCCCCCRICCCCCCCCC
SUSAN	SUSCCCCCCCCCN
MARGARET	MCCCCCCCCCRGCCCCCCCCCRET
LISA	LISCCCCCCCC
NANCY	NCCCCCCCCCNCY



06-1 문자열 함수

■ 공백 문자를 생성하는 함수 — SPACE

- SPACE 함수는 지정한 인자만큼 공백을 생성함
- 다음은 CONCAT 함수로 문자열을 연결할 때
열과 열 사이에 공백 문자 10개를 추가하는 쿼리임

Do it! SPACE 함수로 공백 문자 반복

```
SELECT CONCAT(first_name, SPACE(10), last_name) FROM  
customer;
```

실행 결과

CONCAT(first_name, SPACE(10), last_name)	
MARY	SMITH
PATRICIA	JOHNSON
LINDA	WILLIAMS
BARBARA	JONES
ELIZABETH	BROWN
JENNIFER	DAVIS
MARIA	MILLER
SUSAN	WILSON
MARGARET	MOORE



06-1 문자열 함수

■ 공백 문자를 생성하는 함수 — SPACE

- 앞서 우리는 CONCAT 함수로 열 이름과 문자열을 인자로 사용해 지정한 두 열에 있는 데이터들을 연결할 때 데이터와 데이터 사이에 문자열을 넣어 구분한 적이 있음
- 여기서는 문자열 대신 SPACE 함수를 통해 공백 문자 10개를 넣어 구분한 결과를 확인할 수 있음



06-1 문자열 함수

■ 문자열을 역순으로 출력하는 함수 — REVERSE

- REVERSE 함수는 문자열을 거꾸로 정렬하는 함수임
- 사용할 일이 없을 것 같지만 다양한 문자열 함수와 혼합하면 이메일에서 도메인의 자릿수나 IP 대역을 구할 때 등 다양하게 활용 할 수 있음

1. 다음은 REVERSE 함수를 사용해 문자열을 역순으로 반환해 보자.

Do it! REVERSE 함수로 문자열을 역순으로 반환

```
SELECT 'Do it! MySQL', REVERSE('Do it! MySQL');
```

실행 결과

Do it! SQL	reverse('Do it! SQL')
Do it! SQL	LQS !ti oD

쿼리를 실행한 결과, 'Do it! SQL'을 거꾸로 출력한 'LQS !ti oD'를 확인할 수 있음



06-1 문자열 함수

■ 문자열을 역순으로 출력하는 함수 — REVERSE

2. REVERSE 함수도 여러 함수와 조합하여 사용하면 매우 유용함.
다음 쿼리는 POSITION 함수, CHAR_LENGTH 함수, SUBSTRING 함수를 조합하여 IP 주소의 3번째 부분까지 정보만 조회함

Do it! REVERSE 함수와 다른 여러 함수 조합

```
WITH ip_list (ip)
AS (
    SELECT '192.168.0.1' UNION ALL
    SELECT '10.6.100.99' UNION ALL
    SELECT '8.8.8.8' UNION ALL
    SELECT '192.200.212.113'
)
SELECT ip, SUBSTRING(ip, 1, CHAR_LENGTH(ip) - POSITION('.') IN
REVERSE(ip)))
FROM ip_list;
```




06-1 문자열 함수

■ 문자열을 역순으로 출력하는 함수 — REVERSE

2.

실행 결과	
ip	SUBSTRING(p, 1, CHAR_LENGTH(p) - POSITION('.') IN REVERSE(p))
192.168.0.1	192.168.0
10.6.100.99	10.6.100
8.8.8.8	8.8.8
192.200.212.113	192.200.212

CHAR_LENGTH(ip) 함수를 사용하여 IP 열에 대한 전체 자릿수를 구한 다음, POSITION('.') IN REVERSE(ip)) 함수를 사용하여 IP 열에 대해서 뒤에서부터 가장 먼저 나오는 점(.)까지의 길이를 구함.

마지막으로 SUBSTRNG에서 IP 전체 길이에서 1부터 시작해서 POSITION 길이를 뺀 만큼 문자열을 출력함



06-1 문자열 함수

■ 문자열을 비교하는 함수 — STRCMP

- STRCMP 함수는 두 문자열을 비교하여 동일한지를 알려 줌
- 비교하는 두 문자열이 동일할 경우 0을, 다를 경우 -1을 반환함

1. 다음 쿼리를 실행하면 두 문자열이 동일한 경우이므로 0을 출력함

Do it! STRCMP 함수로 두 문자열을 비교: 동일한 경우

```
SELECT STRCMP('Do it! MySQL', 'Do it! MySQL');
```

실행 결과

STRCMP('Do it! MySQL', 'Do it! MySQL')
0



06-1 문자열 함수

■ 문자열을 비교하는 함수 — STRCMP

2. 다음은 앞선 실습에서 사용한 인자 중 오른쪽 문자열에 !를 추가한 쿼리임.
비교 결과에서 -1을 반환하는 것을 확인할 수 있음

Do it! STRCMP 함수로 두 문자열을 비교: 동일하지 않은 경우

```
SELECT STRCMP('Do it! MySQL', 'Do it! MySQL!');
```

실행 결과

STRCMP('Do it! MySQL', 'Do it! MySQL!')
-1